

This is a copy of a conversation between ChatGPT & Anonymous.
Report conversation

Apologies for the pdf print template,
refer to here for chat log

<https://chatgpt.com/share/693a734e-4384-800d-bf22-287d10d8f5c5>

Here we start with the initial prompt

You are an AI study assistant helping me learn deep learning concepts.

I will provide content from my lecture slides. Your task is to transform the slide content into the most concise, high-yield cheat sheet possible.

Follow these rules carefully:

1. Keep only information that is directly relevant for understanding and applying the topic.
2. Remove all irrelevant, redundant, or overly verbose content. However, DO NOT remove any concept, equation, distinction, or detail that is important for correct understanding.
3. Prioritize concise structure: bullet points, equations, short definitions, and comparison tables.
4. Include:
 - Key equations (LaTeX)
 - Core intuition (2–3 sentences max)
 - When to use each method
 - Common mistakes or misconceptions
5. Everything must fit into a compact study-friendly format — imagine one page of notes.
6. If I request a refinement, rewrite the cheat sheet cleanly incorporating all constraints.

Please paste the lecture slide content.

Uploaded a file Uploaded a file Uploaded a file Uploaded a file
Uploaded a file Uploaded a file

Slides complete. Please generate the cheat sheet according to the rules above.

Upload content

I can generate the cheat sheet—but before I do, I need one crucial clarification:

🔥 Do you want ONE unified cheat sheet covering Lectures 2–7 (all topics), or SEPARATE cheat sheets (one per lecture)?

Because:

- A *single* cheat sheet = extremely compressed, all optimizers + linearization + initialization + NTK + μP + Muon in one page.
- *Multiple* cheat sheets = each lecture gets its own one-page high-yield summary (more complete & easier to study).

Given how dense the slides are, both formats are realistic, but the result will look very different.

✓ Please answer:

Option A — One combined cheat sheet (all lectures 2–7)
Option B — One cheat sheet per lecture (Lecture 2, 1, 5, 6, 7 separately)
Option C — Combine by topic (e.g., "Optimizers", "Initialization", "Linearization/NTK", "Muon")

Once you choose, I will generate a clean, fully distilled cheat sheet with citations to each lecture PDF as required (e.g., Lecture 2).

Which option do you prefer?

Sources

Please generate one unified cheat sheet covering all slides from Lectures 2–7. However, organize by content and not lecture number.

Answer gpt's question

Below is a **single unified, ultra-condensed, high-yield cheat sheet** covering all content in **Lectures 2–7, organized by concept**, not by lecture number.

All information is distilled directly from the uploaded slides (citations included at the end of each section).

★ Deep Learning Optimization & Initialization – Unified Cheat Sheet (Lectures 2–7)

(GD, SGD, Momentum, Adam/AdamW, Linearization & NTK, Initialization, μP , Shampoo/Muon)

1 Gradient Descent (GD) – Core Mechanics

Update

Key intuition

- GD moves in the span of the data; SVD determines step magnitudes.
- Directions with **large singular values** → **large steps**, small singular values → slow progress.
- GD solution for linear regression converges to the **minimum-norm** solution.

Stability

$$\eta \leq \frac{1}{\sigma_{\max}(X^T X)}$$

Implicit regularization

- GD naturally suppresses movement in directions with **small singular values**, similar to ridge regression.
- Early stopping** acts like regularization.

Lecture 2

2 Ridge Regression & Implicit Bias

Ridge solution:

$$w_\lambda = (X^T X + \lambda I)^{-1} X^T y$$

Decomposed in SVD:

$$w_\lambda = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i (v_i^T y)$$

Intuition:

- Large λ kills components along small singular-value directions.
- GD with early stopping behaves similarly because small- σ directions change slowly.

Lecture 2

3 Stochastic Gradient Descent (SGD)

Update

$$w_{t+1} = w_t - \eta \nabla f_{i_t}(w_t)$$

Intuition

- Adds noise $\rightarrow$ can escape local minima/saddle points.
- Cheaper per step; fits subset of data each iteration.
- Converges with decreasing LR or even constant LR when loss $\rightarrow 0$.

SGD only moves in the data span

Because its dynamics depend on X 's SVD, same as GD.

Convergence Sketch

- Define error variable z_t in SVD basis.
- Show expected Lyapunov decrease:

$$\mathbb{E}[\|z_{t+1}\|^2] \leq (1 - c\eta + c'\eta^2) \|z_t\|^2$$

for sufficiently small η .

Lecture 2



4 Momentum

Update (classical)

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla L(w_t), \quad w_{t+1} = w_t - \eta v_{t+1}$$

Benefits

- Smooths oscillations; accelerates along consistent gradient directions.
- Interpretable as **low-pass filtering** of gradients (physics analogy: RC circuit).

Intuition in SVD space:

- GD step sizes depend on σ ; Momentum mitigates slow progress in small- σ directions.

Lecture 2

5 Adam

Adam = Momentum + adaptive step sizes.

Definitions

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(w_t)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(w_t))^2$$

Bias-corrected estimates

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

Update

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Intuition

- Normalizes gradient magnitude $\rightarrow$ better conditioned step sizes.
- Avoids exploding/vanishing gradients.
- May converge poorly without weight decay due to normalization effects.

AdamW

Decouples weight decay:

$$w_{t+1} = (1 - \eta\lambda) w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Lecture 3

6 Sign-SGD (and unified view of optimizers)

Goal:

Constrain the size of the update to prevent local linearization

Formulation:

A unifying principle:

- ## 7 Linearization of Neural Networks (NTK Perspective)

Neural network:

Linearize around current weights:

Here, features = gradients w.r.t. parameters.

Lazy training assumption:

Weights move only slightly $\rightarrow$ linear approximation valid during training.

Neural Tangent Kernel (NTK):

Consequence:

Wide networks $\rightarrow$ NTK nearly constant $\rightarrow$ GD becomes kernel regression and converges.

Lecture 4

- ### Xavier Initialization (tanh/sigmoid)

Goal: keep variance of activations constant.

For fan-in = d :

Variance propagation:

He Initialization (ReLU)

ReLU outputs 0 half the time $\rightarrow$ need doubled variance:

Bias initialization

- Lecture 4

ny It

Motivation: relate optimizer geometry to initialization geometry.

Matrix RMS-norm

This scaling naturally connects to Xavier/He initialization.

Consequence:

Layer-wise learning rate should scale like:

→ key idea behind μP (Maximal Update Parameterization).

width

Goal: make hyperparameters *transferable* across width.

Key ideas:

- RMS→RMS norm gives condition:

$$\eta \sim \frac{1}{d_{in}}$$

Desiderata

- (1) Feature magnitudes stay $O(1)$
- (2) Parameter updates per layer stay $O(1)$

Lecture 6

1 1 Shampoo Optimizer

Uses **matrix preconditioners** $G_t G_t^T$ and their roots to whiten gradients.

Update

$$W_{t+1} = W_t - \eta \left(G_t G_t^T\right)^{-1/2} G_t \left(G_t^T G_t\right)^{-1/2}$$

Core idea: approximate second-order update **per dimension structure**.

Lecture 5

Lecture 7

1 2 Muon Optimizer (Momentum Orthogonalized by Newton–Schulz)

Muon = Momentum + approximate spectral norm $\downarrow$ direction of gradient direction.

Key ideas:

(1) Semi-orthogonal update direction

Instead of using $U \Sigma V^T$, replace singular values with 1:

$$U \Sigma V^T \rightarrow U V^T$$

(2) Newton–Schulz iterations approximate matrix inverse square roots

Given polynomial $p(x)$ that preserves singular vectors:

$$p(U \Sigma V^T) = U p(\Sigma) V^T$$

Apply iteratively so $p(\Sigma) \rightarrow I$.

Practical Muon step

$$B_t = \mu B_{t-1} + \nabla L(W_t)$$

$$O_t = \text{NewtonSchulz}(B_t)$$

$$W_{t+1} = W_t - \eta O_t$$

Why it works:

- Makes update **uniform across singular directions**; avoids domination by the largest σ .
- More stable than Shampoo and cheaper than full SVD.

Lecture 7

1 3 When to Use What (Practical Guide)

Method	When to Use	Key Advantage $\downarrow$	Common Pitfalls
	problems		problems
SGD	large datasets; DL training	noise helps escape saddles	sensitive to LR schedule
Momentum	training unstable; zig-zag loss	smoother trajectory	too much $\beta \rightarrow$ overshooting
Adam	NLP, vision, transformers	adaptive normalization	may fail without AdamW
AdamW	modern DL default	proper weight decay	LR & β interaction still tricky
Sign-SGD	bandwidth-limited, distributed	sign-only communication	convergence issues on noisy gradients
Shampoo	very large models	quasi-second-order	expensive preconditioners
Muon	large-scale LLMs; width-varied nets	stable across layers; SVD-free orthogonalization	polynomial choice & iterations matter

1 4 Most Important Intuitions (1–2 sentences)

- **Optimization = geometry**: every optimizer is solving “best local step under a norm constraint.”
- **Initialization = signal preservation**: choose weight scales so activations/gradients stay $O(1)$ across layers.
- **Linearization/NTK** explains why wide networks $\downarrow$ in like kernel methods.
- **μ P & Muon** exist because hyperparameters must scale with width for stable training.

If you'd like, I can also generate:

- ✓ A one-page printable PDF
- ✓ A mind map
- ✓ A compressed “super cheat sheet” with only formulas & intuitions

Would you like an even more compressed version?

Sources

This cheat sheet is excellent, but I want to refine it further to improve both study efficiency and conceptual understanding.

Self critique

Please do the following:

- 1. Identify any sections that are still too verbose or repetitive, and compress them further without losing any essential meaning.
- 2. Improve internal consistency of notation (e.g., using the same conventions for gradients, updates, matrices).
- 3. Add a final section called "Unifying Principles Across All Topics", where you extract the 4–6 deepest conceptual insights that link optimization, initialization, linearization/NTK, and $\mu P/\mu on$.
- 4. Reorganize the cheat sheet so that related ideas are placed closer together—for example, RMS→RMS norm should appear right before μP , not far away. ↓
- 5. Keep the structure conceptual, not by lecture number.

After generating the revision, provide a brief explanation of the key improvements you made.

Below is the revised, compressed, reorganized, notation-consistent, study-optimized unified cheat sheet.

All citations remain in place.

A short explanation of improvements follows at the end.

★ Unified Deep Learning Optimization & Initialization Cheat Sheet (Lectures 2–7)

(GD, SGD, Momentum, Adam/AdamW, Sign-SGD, Linearization/NTK, Initialization, RMS→RMS Norm, μP , Shampoo, Muon)

1. Core Gradient-Based Optimization Methods

1.1 Gradient Descent (GD)

Update

$$w_{t+1} = w_t - \eta \nabla L(w_t).$$

Key properties

- Moves only in the span of the data (SVD of X ↓ states dynamics).
- Stable & stable.

$$\eta \leq \frac{1}{\sigma_{\max}(X^\top X)}.$$

- Converges to the minimum-norm solution for overparameterized linear models.
- Slower progress in directions with small singular values (implicit bias).

Lecture 2

1.2 Stochastic Gradient Descent (SGD)

Update

$$w_{t+1} = w_t - \eta \nabla \ell_i(w_t).$$

Intuition

- Cheaper per iteration; injects noise → helps escape saddles/local minima.
- Shares GD's geometric constraints (moves in data span).
- Converges if:

$$\mathbb{E}[\|z_{t+1}\|^2] \leq (1 - c\eta + c'\eta^2)\|z_t\|^2.$$

Lecture 2

1.3 Momentum

Update

$$v_{t+1} = \beta v_t + (1 - \beta) \nabla L(w_t), \quad w_{t+1} = w_t - \eta v_{t+1}.$$

Effect

- Low-pass filters gradients → smooths oscillations; accelerates in consistent directions.
- Mitigates slow movement in small-singular-value directions.

Lecture 2

1.4 Adam / AdamW

Moment estimates

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

Bias-corrected

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t}.$$

Update

$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}.$$

AdamW (decoupled weight decay)

$$w_{t+1} = (1 - \eta \lambda) w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t + \epsilon}}.$$

Lecture 3

1.5 Sign-SGD (norm-constrained view)

1. Sign-SGD (norm-constrained view)

Solve:

$$\max_{\|\Delta w\|_\infty \leq \eta} \langle \nabla L(w), \Delta w \rangle.$$

Solution:

$$\Delta w = -\eta \operatorname{sign}(\nabla L(w)).$$

Interpretation: choosing the ∞ -norm constraint yields Sign-SGD; choosing other norms yields GD, Shampoo, Muon, etc.

Lecture 5

2. Implicit Regularization & Ridge Interpretation

Ridge closed form:

$$w_\lambda = (X^\top X + \lambda I)^{-1} X^\top y = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i (v_i^\top y).$$

GD early stopping $\approx$ ridge

- GD naturally under-updates in directions with small singular values.
- Early stopping suppresses high-frequency/low-energy components.

Lecture 2

3. Linearization, Feature Learning, and NTK

3.1 Local linearization

$$f(x, \theta + \Delta\theta) \approx f(x, \theta) + \langle \nabla_\theta f(x, \theta), \Delta\theta \rangle.$$

3.2 Neural Tangent Kernel (NTK)

$$K(x, x') = \langle \nabla_\theta f(x, \theta), \nabla_\theta f(x', \theta) \rangle.$$

Wide networks $\rightarrow$ NTK nearly constant $\rightarrow$ training reduces to convex kernel learning.

Lecture 3

Lecture 4

4. Initialization Principles

4.1 Xavier (tanh/sigmoid)

Maintain activation variance:

$$w_i \sim \mathcal{N}(0, 1/d_{\text{in}}).$$

4.2 He (ReLU)

Compensates for ReLU's $\frac{1}{2}$ survival rate:

$$w_i \sim \mathcal{N}(0, 2/d_{\text{in}}).$$

Goal: keep both activations and gradients $O(1)$ layer-to-layer.

Lecture 4

5. RMS $\rightarrow$ RMS Norm (Optimizer Geometry Aligned With Initialization)

Motivated by wanting an optimizer geometry that matches initialization scaling.

RMS $\rightarrow$ RMS induced matrix norm

$$\|A\|_{\text{RMS} \rightarrow \text{RMS}} = \max_{\|x\|_{\text{RMS}}=1} \|Ax\|_{\text{RMS}} = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|A\|_2.$$

This aligns with Xavier/He scaling: derivatives scale with $1/\sqrt{d_{\text{in}}}$.

Consequence for updates:

To prevent updates from exploding with width:

$$\eta_\ell \propto \frac{1}{d_{\text{in}}}.$$

Lecture 6

6. μP — Maximal Update Parameterization

Purpose: hyperparameters stable across width.

Requirements

- Activations remain $O(1)$ as width increases.
- Updates ΔW_ℓ remain $O(1)$.
- Feature learning does not collapse at large width.

Scaling rule from RMS $\rightarrow$ RMS norm:

$$\eta_\ell \sim \frac{1}{d_{\text{in}}}, \quad \|\Delta W_\ell\|_{\text{RMS} \rightarrow \text{RMS}} = O(1).$$

Effect:

- Prevents NTK limit from dominating when feature learning is desired.

Lecture 6

7. Second-Order Approximations: Shampoo

Uses Kronecker-factored approximate curvature (per dimension).

Update (no accumulation version shown in slides)

Update the accumulation version shown in slides

$$W_{t+1} = W_t - \eta (G_t G_t^\top)^{-1/2} G_t (G_t^\top G_t)^{-1/2}.$$

Key idea: approximate second-order whitening in input/output dimensions.

- Lecture 5
- Lecture 7

8. Muon Optimizer (Momentum Orthogonalized by Newton–Schulz)

Muon = Momentum + approximate orthogonalization of the update direction using Newton–Schulz iterations.

Step 1 — Momentum in gradient space

$$B_t = \mu B_{t-1} + g_t.$$

Step 2 — Use Newton–Schulz to compute approximate

$$\tilde{O}_t \approx UV^\top \quad (\text{singular values} \rightarrow 1).$$

$$p(U\Sigma V^\top) = U p(\Sigma) V^\top, \quad p(\Sigma) \rightarrow I.$$

Step 3 — Update

$$W_{t+1} = W_t - \eta \tilde{O}_t.$$

Why it works

- Removes the effect of large singular values dominating updates.
- Creates **uniform, geometry-consistent** steps across layers.
- Cheaper than exact SVD; robust at LLM scale.

- Lecture 7

9. When to Use Each Method (Ultra-Condensed)

Method	Best Use	Key Strength
GD	small/convex	clean geometry
SGD	large data	noise → exploration
Momentum	oscillatory landscapes	smooths trajectory
AdamW	transformers, deep nets	adaptive + correct decay
Sign-SGD	comm.-limited	sign-only updates
Shampoo	very large models	quasi-2nd-order

$$M_t = \frac{1}{\sqrt{d}} \sum_{i=1}^d \frac{g_i}{\|g_i\|} \otimes \frac{g_i}{\|g_i\|}$$

★ 10. Unifying Principles Across All Topics (Most Important Insights)

1. Optimization = Choosing a Geometry

Every optimizer solves “best local step under a norm constraint.”
Different norms (2-norm, ∞-norm, spectral norm, RMS→RMS) yield GD, Sign-SGD, Shampoo, Muon, etc.

2. Initialization and Optimization Must Match

Xavier/He ensure activations/gradients stay O(1).
Muon, Shampoo, μP construct **update geometries** consistent with these scaling laws.

3. SVD/curvature structure governs all dynamics

Implicit bias, convergence speed, GD/SGD path, momentum acceleration, and Muon orthogonalization are all consequences of how singular values shape allowable movement.

4. Width fundamentally changes training behavior

Wide networks → NTK regime (lazy training).
μP and Muon restore meaningful feature learning by scaling updates correctly.

5. Noise and curvature jointly determine optimization success

SGD noise aids exploration; curvature-aware methods (Shampoo, Muon) fix poor conditioning; momentum smooths unstable curvature.

6. Hyperparameter transferability requires correct scaling

μP provides the universal rule: scale learning rate by fan-in to maintain update sizes and escape NTK

✔ Summary of Improvements Made

Compression

- Removed redundant phrasing; every section now contains only essential equations + intuition.
- Consolidated GD/SGD/momentum links to SVD-based geometry.

Notation Consistency

- Uniform use of $w_t, g_t = \nabla L(w_t), B_t$ for momentum buffer, $d_{\text{in}}, d_{\text{out}}$.

Reorganization

- Grouped RMS→RMS norm directly before μP.
- Clustered Shampoo and Muon (geometry-based optimizers) together.
- Placed implicit regularization next to GD/SGD rather than between unrelated material.

New Section

- Added **Unifying Principles** synthesizing optimization, initialization, NTK, μP, and Muon into deep conceptual takeaways.

If you want, I can now:

- ✓ Produce a **one-page printable PDF** version
- ✓ Add diagrams summarizing relationships between optimizers
- ✓ Produce an **"equations only" micro-cheat sheet** ↓

What do you think of this cheat sheet?

Sources

Great — now please perform a critical self-review of the cheat sheet:

1. Identify any subtle inaccuracies, oversimplifications, or places where the explanation may mislead a student.
2. Identify any missing concepts from the slides that should be included for correctness.
3. Highlight any equations that might require additional conditions or assumptions.
4. Point out any places where my understanding might suffer if I rely only on the cheat sheet.
5. Suggest 3–5 clarifying questions I should ask myself to test deep understanding of these topics.
6. Suggest 3–5 ways I could actively use this cheat sheet to reinforce long-term retention (spaced repetition, retrieval cues, diagrams, etc.).

Do NOT rewrite the cheat sheet yet — only provide the critique and the study recommendations.

Self critique

Below is a **critical, rigorous self-review** of the cheat sheet you asked for.

This is **NOT a rewrite** — it is a detailed audit of correctness, omissions, assumptions, and pedagogical risks, plus study recommendations.



✓ I. Subtle inaccuracies or misleading oversimplifications

(a) "GD converges to the minimum-norm solution for overparameterized linear models"

✓ True *only if*:

- Initialization is at zero (or in the nullspace of X), and
- The step size stays within stability bounds, and
- Training converges fully to zero loss.

The cheat sheet states this as unconditional.

(b) "SGD shares GD's geometric constraint (moves in the span of the data)"

✓ Correct for *linear* models or first-order Taylor approximations.

⚠ But for **nonlinear networks**, SGD moves in the span of **backpropagated gradients**, which depend on activations and parameters, not just the raw data matrix.

This may mislead a student into thinking SGD is strictly limited to the input data span.

(c) Treatment of implicit regularization

The cheat sheet correctly states GD and early stopping mimic ridge but:

- The *exact equivalence* only holds for **gradient flow** or infinitesimal step sizes.
- For discrete GD, the relationship is approximate unless step size $\rightarrow 0$.

(d) Momentum "mitigates slow movement in small singular-value directions"

1. Missing Concept

converge slowly unless the momentum parameter is carefully tuned.

(e) Adam section glosses over generalization issues

Adam can achieve zero training loss but generalize much worse than SGD, especially on vision tasks.

Cheat sheet implies AdamW "fixes" Adam's issues, which is only *partially* true.

(f) RMS→RMS norm formula correctness

The relation:

$$\|A\|_{RMS \rightarrow RMS} = \sqrt{\frac{d_{in}}{d_{out}}} \|A\|_2$$

is correct **under the definition of RMS norm used in the slides**, but is *not* a universal identity.

Some students may assume a general operator norm identity.

(g) μP scaling rule " $\eta \sim 1/d_{in}$ " is heuristic

This is the **leading-order behavior**, but the actual μP framework uses:

- precise scaling for weights, biases, and learning rates,
- distinctions between feature and readout layers,
- assumptions about width $\rightarrow \infty$ limits.

The cheat sheet simplifies this into a single line.



(h) Muon "orthogonalizes the update direction"

This is **approximately true**, but Muon does not produce a perfectly orthonormal direction; it uses a truncated Newton–Schulz iteration and may not converge fully.

Student might think Muon literally performs full orthogonalization.

(i) Shampoo update formula missing the ϵ -stabilizers

Shampoo implementations always include stabilizing ϵ terms to ensure invertibility; slides may omit these for clarity, but students should know this.

✔ 2. Concepts present in the slides but absent or underemphasized

(a) Detailed derivation of SGD convergence using Lyapunov function

Slides show:

- decomposition of update into A and B terms
- expected contraction
- dependence on sampling randomness

The cheat sheet summarises this but does not convey:

- why the A term contracts
- why the B term remains controlled
- the role of full-rank assumptions.

Message CheatSheet
Lecture 6/7 emphasizes:

- for single sample gradient, $g = uv^T$ is rank-1
- crucial for Muon and norm scaling reasoning.

Cheat sheet does not explicitly mention this footprint.

(c) Feature learning desiderata in μP

Slides highlight conditions:

- hidden activations remain $O(1)$
- ΔW remain $O(1)$
- norms not negligible across width

These were summarized but not systematically enumerated.

(d) Shampoo derivation using SVD and Kronecker structure

Slides explicitly show how:

$$G_t G_t^\top = U \Sigma^2 U^\top$$

and similar expressions.

Cheat sheet only sketches the final formula.

(e) Conditions for NTK regime vs feature learning regime

Slides delineate when training stays close to initialization vs when it learns features.

✔ 3. Equations needing assumptions/conditions

(a) GD stability bound

$$\eta \leq \frac{1}{\sigma_{\max}(X^\top X)}$$

holds for squared loss; for general losses, Lipschitz gradient assumptions required.

(b) Ridge-to-GD equivalence

Depends on minimum-norm initialization, convex quadratic loss, and convergence to zero-loss solution.

(c) NTK linearization

$$f(\theta + \Delta\theta) \approx f(\theta) + J\Delta\theta$$

is only valid when parameter changes are small, i.e., wide nets or small learning rates.

(d) Newton–Schulz iteration

Convergence requires singular values of the normalized matrix to lie within $(0,1)$; slides mention normalizing by Frobenius norm, which cheat sheet omitted.

(e) $\ell_2 \rightarrow \ell_2$ RMS operator norm identity

Uses a specific RMS definition from slides, not universal.

✔ 4. Where student understanding may suffer if relying only on cheat sheet

(a) Lack of intuition about why μP works

Cheat sheet states scaling rules but does not explain the *core principle*:

Keep the **function update** (not weight update) invariant with width.

Without this, μP feels like a rule, not a theory.

(b) Insufficient detail on NTK vs feature learning transition

Understanding that NTK $\neq$ general learning is key.
Cheat sheet suggests NTK is typical for wide nets but does not highlight how μP and Muon deliberately prevent NTK collapse.

(c) Missing visualization intuition for Shampoo/Muon

The slides include diagrams showing trajectories with and without momentum/orthogonalization.
Cheat sheet loses these geometric insights.

(e) Adam's generalization pitfalls

Cheat sheet presents AdamW positively; a student may miss when SGD is preferable.

5. Clarifying questions to test deep understanding

- 1. Why does gradient descent converge to the minimum-norm solution in overparameterized linear regression, and what assumptions are necessary?
- 2. What conditions guarantee that the NTK linearization is accurate throughout training?
- 3. Explain how the RMS $\rightarrow$ RMS norm leads to μP scaling. Why does fan-in appear naturally?
- 4. Why do Muon and Shampoo improve conditioning? How do they relate to the curvature of the loss landscape?
- 5. When does Adam generalize worse than SGD? What role does adaptive normalization play in this difference?

6. Techniques to actively use the cheat sheet for long-term retention

(a) Spaced retrieval prompts

Create flashcards with:

- definitions (e.g., "What is NTK?"),
- conceptual contrasts (e.g., GD vs momentum vs Adam),

(b) Diagram reconstruction

Re-draw from memory:

- GD vs momentum trajectories,
- Muon orthogonalization pipeline,
- NTK tangent space diagram,
- gradient rank-1 structure.

(c) "Explain without equations" drills

Practice summarizing:

- Why momentum helps,
- Why initialization scaling matters,
- Why μP avoids NTK collapse.

This strengthens conceptual reasoning.

(d) Memory palaces / chunking by theme

Group topics by their "geometry":

- **Data geometry** $\rightarrow$ GD, SGD, ridge
- **Curvature geometry** $\rightarrow$ momentum, Shampoo
- **Width geometry** $\rightarrow$ initialization, RMS $\rightarrow$ RMS, μP , Muon
- **Representation geometry** $\rightarrow$ NTK

(e) Generation exercises

Force yourself to derive:

- GD stability bound,
- μP scaling from RMS $\rightarrow$ RMS,
- one step of Newton-Schulz normalization.

This deepens procedural knowledge.

If you'd like, I can now:

- ✓ Patch every inaccuracy and integrate corrections into a **final improved cheat sheet**,
- OR
- ✓ Generate a "what the slides *really* mean" commentary version.

Which would you prefer next?

Please generate a corrected and improved final version of the unified cheat sheet.

Incorporate all issues you identified in your self-review:

- fix inaccuracies and misleading statements,
- add missing concepts,
- clarify assumptions for all equations,
- expand areas where student intuition may fail,
- and ensure notation is consistent.

Keep the cheat sheet concise and study-optimized, but ensure conceptual correctness and completeness.

Generate after Self critique

1

Unified Deep Learning Optimization, Initialization & Scaling Cheat Sheet (Corrected & Improved)

It incorporates every issue identified in the self-review while remaining compact and study-friendly.

1

Unified Deep Learning Optimization, Initialization & Scaling Cheat Sheet (Corrected & Improved)

(GD, SGD, Momentum, Adam/AdamW, Sign-SGD, NTK/Lazy Training, Initialization, RMS→RMS Norm, μP , Shampoo, Muon)

0. Notation (Consistent Across Sheet)

- Parameters $w \in \mathbb{R}^d$.
- Loss $L(w)$, sample loss $\ell_i(w)$.
- Gradient $g_t = \nabla L(w_t)$; minibatch gradient $g_{it} = \nabla \ell_{it}(w_t)$.
- For matrices (e.g., layer weights) use W , gradients G .
- SVD: $A = U \Sigma V^\top$.
- Fan-in = input dimension d_{in} ; fan-out = output dimension d_{out} .

1. First-Order Optimization

1.1 Gradient Descent (GD)

$$w_{t+1} = w_t - \eta g_t.$$

Conditions for stability

$$\text{For } L(w) = \frac{1}{2} \|Aw - y\|^2.$$

$$\eta \leq \frac{1}{\lambda_{\max}(X^\top X)}.$$

For general smooth losses: require **L-smoothness** (gradient Lipschitz constant L), then

$$\eta < \frac{2}{L}.$$

Geometry (under linear model)

- GD updates lie in the **column space of X** .
- Converges to **minimum-norm interpolating solution** *only if*
 - initialization lies in $\ker(X)$ (e.g., $w_0 = 0$),
 - step size is stable,
 - loss reaches zero.

Implicit bias

- GD suppresses movement in small-singular-value directions → resembles ridge penalty.
- Exact equivalence holds for **gradient flow**; discrete GD is approximate.

Lecture 2

1.2 Stochastic Gradient Descent (SGD)

$$w_{t+1} = w_t - \eta g_{it}.$$

Key facts

- Noisy updates help escape saddle points.
- Converges under Robbins–Monro conditions (decaying LR) or with constant LR if zero-loss.
- For linear models, SGD also stays in **data span**;
for nonlinear nets, updates lie in span of **backpropagated activations**, not raw data.

Convergence idea from slides

Let z_t be error in SVD coordinates. Show

$$\mathbb{E} \|z_{t+1}\|^2 \leq (1 - c\eta + c'\eta^2) \|z_t\|^2$$

by splitting the update into a contracting linear term (A) and bounded noise term (B).

Assumes full-rank data and small enough η .

Lecture 2

1.3 Momentum

$$v_{t+1} = \beta v_t + (1 - \beta) g_t, \quad w_{t+1} = w_t - \eta v_{t+1}.$$

Behaves as **low-pass filtering** of gradients; accelerates along consistent curvature directions.
Does not fix severe conditioning by itself—still limited by the smallest Hessian eigenvalues.

Lecture 2

1.4 Adam / AdamW

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$w_{t+1} = w_t - \eta \frac{m_t / (1 - \beta_1^t)}{\sqrt{v_t / (1 - \beta_2^t) + \epsilon}}.$$

- Adaptive scaling improves conditioning.
- AdamW** fixes weight decay by decoupling from the adaptive update.
- Generalization can be worse than SGD (esp. vision tasks).

Lecture 3

1.5 Sign-SGD (Norm-Constrained View)

Solve:

$$\min_{\|\Delta w\|_\infty \leq \eta} \langle g_t, \Delta w \rangle \Rightarrow \Delta w = -\eta \text{sign}(g_t).$$

Highlights the principle: **changing the update-norm constraint yields different optimizers.**

Lecture 5

2. Ridge Regression & Implicit Regularization

Ridge closed form

$$w_\lambda = (X^\top X + \lambda I)^{-1} X^\top y = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i (v_i^\top y).$$

GD connection

- Gradient flow exactly equals ridge with time-dependent λ .
- Finite-step GD approximates this only with small η .
- Early stopping suppresses small-singular-value components.

Lecture 2

3. Linearization & NTK

3.1 First-order Taylor expansion

$$f(x, \theta + \Delta \theta) \approx f(x, \theta) + \nabla_\theta f(x, \theta)^\top \Delta \theta.$$

Valid *only* when parameter movement is small — typical for **very wide networks** or very small LR.

3.2 Neural Tangent Kernel (NTK)

$$K(x, x') = \nabla_\theta f(x, \theta_0)^\top \nabla_\theta f(x', \theta_0).$$

In infinite-width limit:

- Parameters barely move → training is **kernel regression** with fixed NTK.
- No feature learning; expressivity fixed at initialization.

Lecture 3

Lecture 4

Transition intuition

Small width → nonlinear feature learning.

Large width + standard parameter scaling → NTK/lazy regime.

μP and Muon exist to **escape the lazy limit**.

4. Initialization: Maintaining Stable Variances

4.1 Xavier Initialization (tanh/sigmoid)

$$w_{ij} \sim \mathcal{N}\left(0, \frac{1}{d_{\text{in}}}\right).$$

4.2 He Initialization (ReLU)

Compensates for ReLU's $\frac{1}{2}$ activation survival.

$$w_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right).$$

Goal: maintain forward and backward signal magnitudes **O(1)** across layers; sets the stage for μP and Muon scaling.

Lecture 4

5. RMS→RMS Norm (Optimizer Geometry Aligned With Initialization)

RMS norm

For vector $x \in \mathbb{R}^d$:

$$\|x\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_i x_i^2}.$$

Induced matrix RMS→RMS norm (from slides)

$$\|A\|_{\text{RMS} \rightarrow \text{RMS}} = \max_{\|x\|_{\text{RMS}}=1} \|Ax\|_{\text{RMS}} = \sqrt{\frac{d_{\text{in}}}{d_{\text{out}}}} \|A\|_2.$$

This identity depends on the RMS definition used in the slides; not universal.

very important

Both initialization (Xavier/He) and μP rely on scaling related to fan-in/out.

Thus the **optimizer should respect that geometry**.

Lecture 6

6. μP — Maximal Update Parameterization

Goal: **make hyperparameters width-invariant** and preserve **feature learning** instead of collapsing to NTK.

Core principle

Maintain **O(1)** function updates as width $\rightarrow \infty$.

Conditions (from slides)

1. Activations remain $O(1)$.
2. Weight updates $\|\Delta W_\ell\|_{\text{RMS} \rightarrow \text{RMS}} = O(1)$.
3. Feature updates do not vanish.

Scaling rules (simplified but correct)

$\eta_t \propto \frac{1}{d_{\text{in}}}, \quad W_t \text{ initialization} \propto \frac{1}{\sqrt{d_{\text{in}}}}.$

These rules reproduce the small-movement regime needed for transferable hyperparameters *without* falling fully into NTK.



Lecture 6

7. Second-Order Geometry Methods: Shampoo & Muon

7.1 Shampoo (conceptual)

Approximates curvature by factoring the gradient matrix.
Full (non-accumulating) update from slides:

$$W_{t+1} = W_t - \eta \left(G_t G_t^\top\right)^{-1/2} G_t \left(G_t^\top G_t\right)^{-1/2}.$$

Assumptions:

- Preconditioners require el for invertibility (omitted in slides).
- True implementation maintains running statistics; slides present simplified form.

Lecture 5

Lecture 7

7.2 Muon Optimizer — Momentum Orthogonalized by Newton–Schulz

Step 1 — Momentum accumulation

$B_t = \mu B_{t-1} + G_t.$

For minibatch=1, slides show $G_t = uv^\top$ (rank-1); key to Muon's efficiency.

Step 2 — Normalize direction

Goal: remove singular value distortion:

$$B_t = U \Sigma V^\top \Rightarrow U \Sigma V^\top \mapsto U V^\top.$$

Use polynomial $p(\cdot)$ with property:

$$p(U \Sigma V^\top) = U p(\Sigma) V^\top,$$

with $p(\Sigma) \rightarrow I$. Requires singular values normalized to (0,1). Slides mention normalizing by Frobenius norm before applying.

Step 3 — Update

$$W_{t+1} = W_t - \eta \tilde{O}_t,$$

where $\tilde{O}_t \approx UV^\top$ after few Newton–Schulz iterations.

Intuition

- Removes directional bias from large singular values.
- Produces width-robust step sizes (natural partner to μP).
- Computationally cheaper than exact SVD; more geometry-aware than Adam.

Lecture 7

8. Unifying Principles (Deep Concepts Across All Topics)

1. Optimization = Geometry + Norm Choice

Choosing a constraint set (2-norm, ∞ -norm, spectral norm, RMS→RMS) defines GD, Sign-SGD, Shampoo, Muon.

The optimizer's geometry must match the problem's geometry.

2. Initialization Sets the Scale: Optimizer Must Respect It

Xavier/He ensure activations/gradients stay stable.

μP and Muon ensure updates stay stable and meaningful across widths.

3. SVD/Hessian Structure Dictates Training Dynamics

Implicit bias, conditioning, momentum acceleration, and Muon normalization all originate from how singular values shape allowable movement.

4. Width Controls the Learning Regime

Standard scaling → NTK (lazy) regime.

μP and Muon modify scaling so networks learn features, not only fit via fixed kernels.

5. SGD Noise + Curvature Determine Optimization Landscape Navigation

Noise helps exploration; geometry-aware optimizers improve conditioning; momentum smooths stochastic curvature.

6. Hyperparameter Transfer Requires Correct Scaling

Without μP -style scaling, LR and initialization choices do not generalize across widths.

9. When to Use Which Method (Corrected Summary)

Method	Best For	Strength	Caveat
GD	small convex problems	exact geometry	slow for ill-conditioned cases
SGD	large datasets	exploration via noise	sensitive to LR schedule
Momentum	oscillatory curvature	smooths path	still limited by conditioning
Sign-SGD	distributed/low-bandwidth	low communication cost	fragile to noisy gradients

Shampoo	very large models	quasi-second-order	heavy memory; needs ϵ -stabilizers
Muon	LLM-scale training	width-stable, geometry-consistent	polynomial choice affects behavior

✔ This version is now corrected, assumption-aware, and conceptually unified.

If you'd like, I can now produce either:

- ✂ a one-page printable PDF,
- ✂ a minimal equations-only cheat sheet, or
- ✂ a diagram-based visual summary.

Which format would help you most?

Sources

Now generate 3 challenging diagnostic questions based strictly on the corrected cheat sheet and the issues highlighted in your self-

These questions should be designed to:

- reveal subtle misunderstandings I might have,
- test deep conceptual reasoning (not memorization),
- target areas students commonly get wrong (e.g., assumptions, NTK limits, μP scaling, curvature effects).

Do NOT provide answers yet.

After I respond with my answers, you will:

1. Identify any gaps, confusions, or misconceptions in my reasoning,
2. Explain why these misunderstandings matter,
3. Update or annotate the cheat sheet to address my specific gaps.

Generate questions to test my understanding and identify gaps

Here are 3 challenging diagnostic questions crafted to expose subtle misunderstandings, hidden assumptions, and deep conceptual gaps related to optimization geometry, NTK limits, μP scaling, and curvature effects.

Do NOT answer them yet — wait until you respond first.

? Diagnostic Question 1 — NTK vs Feature Learning Transition

A colleague claims:

"If a network is wide and initialized with Xavier or He initialization, then training will automatically stay in the NTK regime unless we explicitly modify the architecture or add special regularization."

Is this statement correct?

- the assumptions behind NTK linearization accuracy,
- how parameter scaling interacts with width,
- how μP and learning-rate scaling influence whether the network remains in the lazy regime or learns features.

Your explanation should address both **why** the NTK regime emerges and **what must change** for a network not to fall into it.

? Diagnostic Question 2 — Geometry of Updates & μP Scaling

Suppose a fully connected network layer uses weights $W \in \mathbb{R}^{d_{out} \times d_{in}}$, initialized with He initialization. You increase the width by a factor of k , leaving the learning rate η unchanged.

Explain in detail why this may break hyperparameter transferability, referencing:

- RMS→RMS norm scaling,
- the requirement that function updates remain $O(1)$,
- the distinction between weight update magnitude and function update magnitude,
- what happens to gradient scales when width increases.

Be explicit about which assumptions must hold for μP scaling rules to guarantee stable updates across widths.

? Diagnostic Question 3 — Curvature, Momentum, and Muon Orthogonalization

Consider the following claim:

"Momentum can give poor conditioning because it accelerates progress along all gradient directions, so there is no need for methods like Shampoo or Muon unless you want faster convergence."

Evaluate this claim critically, describing:

- how curvature (Hessian eigenvalues/singular values) affects GD and momentum,
- which parts of the optimization dynamics momentum improves *and which it does not*,
- why Muon's approximate orthogonalization changes the update geometry in ways momentum cannot,
- and how second-order structure appears in Shampoo vs the rank-1 gradient structure underlying Muon.

Clarify the underlying assumptions that make certain methods succeed or fail.

When you submit your answers, I will:

✔ Diagnose misunderstandings

- ✓ Explain why they matter
- ✓ Update or annotate the cheat sheet to address your specific reasoning gaps
- Take your time — these are deliberately difficult.

question 1:

i think the statement is partially right but not entirely. wide networks with standard Xavier/He init do tend to fall into the NTK regime because the parameter updates become small relative to the scale of the weights as width grows, so the linear approximation stays accurate.

key part is really how the learning rate scales with width. if we keep η fixed while making the network wider, the effective update size shrinks, so features don't change much and we stay in the lazy/NTK regime. μP changes that because it rescales the learning rate (and also the initialization in some layers) so that the function updates stay $O(1)$. that's what lets feature learning survive even at large width. so i guess the colleague's statement misses the idea that it's the parameterization and lr scaling—not just width alone—that determines whether we fall into NTK. but i'm not sure i fully get the boundary: like, exactly how small the movement has to be, or how to quantify when linearization breaks down.

question 2:

if i increase width by k but keep the same lr, i think the gradients scale differently because the activations get averaged over more units. with He init the variance of activations stays $O(1)$, but the gradient norms don't necessarily stay the same; i think they shrink with width? so the weight update might be "small enough" but the actual function update can change in unpredictable ways. μP tries to make the RMS→RMS update per layer independent of width by dividing lr by fan-in, but i'm not 100% clear on how the RMS→RMS norm directly connects to the functional change—just that the scaling keeps ΔW roughly stable. if we ignore this scaling and just widen the layer, updates might become too small and the model behaves more like NTK, or they might blow up depending on the layer shapes. so hyperparams don't transfer. but i'm still a bit fuzzy on exactly how the gradient magnitude changes when width increases.

question 3:

momentum helps stabilize updates in directions of consistent curvature, but for ill-conditioned problems (where smallest and largest Hessian eigenvalues are really far apart) i don't think momentum actually fixes the slow directions; it mainly boosts the fast ones. so poor conditioning still hurts GD+momentum. Muon does something momentum can't: it sort of removes the effect of the singular values by normalizing the update direction, so each direction gets treated more equally. that's different from Shampoo too, which uses approximate second-order info. Muon relies on the rank-1 structure of gradients (in the minibatch 1 case) so the SVD-like normalization is efficient. momentum doesn't use curvature information at all, so it can't reshape the geometry. but i'm not fully sure how strong the orthogonalization effect is, or how close Muon gets to actually flattening curvature.

My response

Excellent — your answers show strong intuition, but there are **three subtle conceptual gaps** that matter for mastery, especially when applying NTK/ μP reasoning to real architectures or when interpreting Muon's geometry.

Below, I'll give:

1. A precise diagnosis of gaps,

2. Why they matter,

3. Targeted cheat-sheet updates to eliminate these gaps.

1. Diagnostic Feedback for Question 1 (NTK vs Feature Learning)

Where your reasoning is correct

- You correctly emphasized that **learning-rate scaling** and **parameterization** determine whether updates remain small enough for NTK linearization.
- You understood μP 's role: maintaining **$O(1)$ functional updates**, not $O(1)$ weight updates.

Subtle gap 1 — misunderstanding which quantity must stay small

You wrote:

"parameter updates become small relative to the scale of the weights as width grows, so the linear approximation stays accurate."

This is almost right, but incomplete.

The NTK regime is governed by small **functional** updates, i.e.:

$$\Delta f(x) = \nabla_{\theta} f(x, \theta) \cdot \Delta \theta \quad \text{must be } o(1)$$

—not by weight update size per se.

- Even if $\Delta \theta$ is not tiny, NTK may still hold if **features** remain unchanged.
- Conversely, very small $\Delta \theta$ can still break NTK if gradients scale unfavorably.

So the colleague's claim misses the central condition:

$$\|\Delta f\| \ll 1 \quad (\text{lazy regime})$$

Subtle gap 2 — missing the role of **variance-preserving initialization** in lazy training

You implicitly assumed Xavier/He $\rightarrow$ NTK.

The nuance:

- Xavier/He produce **stable preactivation variance** $\downarrow$
- but they do **NOT** guarantee lazy training.

GPT gives feedback

Message ChatGPT

width.

Without LR shrinking, Xavier/He can still produce **feature-learning**, not NTK.

Subtle gap 3 — uncertainty about the “boundary” of linearization validity

You wrote:

“not sure exactly how small the movement has to be, or how to quantify break-down.”

The formal condition is:

$$\|\theta_t - \theta_0\|_2 \ll \frac{1}{\sqrt{\text{width}}}$$

AND equivalently:

$$\|\Delta f\| \ll 1.$$

This boundary explains exactly when NTK fails.

! Why these gaps matter

- Misunderstanding “weight update small vs. functional update small” leads to confusion about μP ’s purpose.
- The failure to articulate exact NTK validity conditions makes it hard to reason about width scaling or LR tuning.
- Without recognizing the distinction between stable activations vs lazy training, you may misattribute phenomena in large-scale training.

Cheat-Sheet Improvements (embedded later)

- Add explicit NTK validity conditions.
- Add distinction between weight-update scale and function-update scale.
- Clarify that Xavier/He do not *force* NTK, only permit it if LR shrinks.

2. Diagnostic Feedback for Question 2 (μP Scaling & RMS→RMS Geometry)

Where your reasoning is correct

- You recognized that keeping LR fixed breaks hyperparameter transferability.
- You noted gradient scales change with width.
- You correctly said μP maintains **functional** update invariance.

Subtle gap 1 — Misunderstanding how gradient magnitudes scale

You wrote:

“i think gradients shrink with width? ... still a bit fuzzy on exactly how the gradient magnitude changes.”

This is a critical detail.

Correct scaling behavior:

If activations stay $O(1)$, and weights follow He/Xavier:

- Gradient **per-parameter** is $O(1/\sqrt{d_{in}})$.
- But total gradient *matrix* norm scales as:

Message ChatGPT

Thus, widening a layer while keeping LR fixed:

- Causes **functional updates to shrink** (lazy regime),
- OR
- Causes **instability** if the wrong dimension is scaled.

The cheat sheet needs to show this explicitly.

Subtle gap 2 — RMS→RMS norm and function updates

You wrote:

“not 100% clear how RMS→RMS connects to function change.”

Key link:

$$\Delta f = J \Delta \theta$$

and the Jacobian J has RMS scaling determined by **fan-in**.

μP ensures:

$$\|\Delta f\| \text{ remains } O(1) \iff \|\Delta W_\ell\|_{RMS \rightarrow RMS} = O(1).$$

So the RMS→RMS norm defines the **exact scaling** required for functional invariance.

! Why this matters

Without understanding gradient scaling:

- You may misinterpret training instability or failure to transfer LRs across widths.
- This knowledge is crucial for reasoning about very deep or very wide networks.

Cheat-Sheet Improvements Needed

- Add explicit gradient scaling formulas with width.
- Clarify RMS→RMS → functional change equivalence.
- Explain precisely where width enters the Jacobian scale.

3. Diagnostic Feedback for Question 3 (Curvature, Momentum, Muon)

Where your reasoning is strong

- You correctly identified that momentum does not fix poor conditioning.
- You understood Muon neutralizes singular value distortion.
- You distinguished Shampoo's curvature approximation from Muon's rank-1 geometry.

Subtle gap 1 — Momentum's exact behavior under curvature

You wrote:

"momentum boosts fast directions but not slow ones."

More precisely:

- In quadratic problems, momentum multiplies eigendirection by:

- It helps when eigenvalues are clustered but **cannot eliminate the condition number**.

So momentum speeds up convergence only when the spectrum is "nice."

Muon/Shampoo reshape the spectrum; momentum does not.

Subtle gap 2 — Overestimating Muon's orthogonalization strength

You wrote:

"not fully sure how close Muon gets to actually flattening curvature."

Important nuance:

- Muon **does not flatten curvature** like second-order Newton methods.
- It **normalizes the gradient direction**, not the Hessian.
- It ensures each singular direction of the gradient contributes equally, which stabilizes width scaling and improves conditioning *indirectly*.

Muon $\neq$ Newton, and this misconception can lead to overestimating its abilities.

! Why this matters

- Misunderstanding "curvature vs gradient singular values" leads to incorrect expectations about Muon versus true second-order methods.
- It's critical to know precisely *what* geometric distortions Muon fixes.

Cheat-Sheet Improvements Needed

- Add explicit statement that Muon does **not** invert curvature.
- Add explanation of momentum's eigendynamics.

Cheat-Sheet Updates (Precise Additions)

Here are explicit additions that will be integrated into the revised cheat sheet (you do not need to rewrite now):

Add under NTK section

$$\|\Delta f\| \ll 1 \quad (\text{lazy regime})$$

and

$$\|\theta_t - \theta_0\|_2 \ll \frac{1}{\sqrt{\text{width}}}.$$

Add under μP

Gradient scaling with width:

$$\|G\|_F = O\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right), \quad G_{ij} = O(1/\sqrt{d_{\text{in}}}).$$

μP ensures:

$$\Delta f = O(1) \forall \text{ width}.$$

Add under Momentum

Momentum modifies eigenvalues but leaves condition number essentially unchanged.

Muon normalizes **gradient singular values**, not Hessian eigenvalues.

It improves stability but is not a curvature-correcting method like Newton/Shampoo.

🎯 If you want, I can now:

Option A — Produce the fully revised cheat sheet with all these corrections embedded,

Option B — Produce a "meta cheat sheet" summarizing the conditions and assumptions,

Option C — Produce diagrams that visually illustrate NTK vs feature learning, gradient scaling, Muon normalization.

Which option would you like next?

Please generate the final revised cheat sheet.

Use all the feedback, corrections, critiques, and reasoning-gap diagnoses from our earlier steps.
This includes:
- fixing inaccuracies and oversimplifications,
- adding any missing important content.

Prompt to generate cheat sheet to incorporate my gaps

- clarifying assumptions and conditions behind formulas and concepts,
- improving explanations where my earlier answers showed gaps,
- refining organization, notation consistency, and conceptual flow,
- and making the cheat sheet concise, accurate, and study-friendly.

Unified Cheat Sheet

previous concerns and improvements.

Below is the final, corrected, assumption-aware, conceptually complete, notation-consistent, and study-optimized unified cheat sheet.

It incorporates *all* issues from the self-review and all gaps revealed in your diagnostic answers.

This is now a **rigorously accurate, concise, and pedagogically optimized** reference for GD/SGD, momentum, AdamW, Sign-SGD, NTK, initialization, RMS→RMS, μP , Shampoo, and Muon.

★ Unified Deep Learning Optimization, Initialization & Scaling Cheat Sheet

(Corrected + Assumptions Explicit + Geometry-Aligned + NTK/ μP Ready)

0. Notation (Unified Across Sheet)

- Parameters: $w \in \mathbb{R}^d$ or weight matrices $W \in \mathbb{R}^{d_{out} \times d_{in}}$.
- Loss: $L(w)$; sample loss: $\ell_i(w)$.
- Gradient: $g_t = \nabla L(w_t)$; minibatch gradient: $g_{b_t} = \nabla \ell_{b_t}(w_t)$.
- SVD: $A = U \Sigma V^\top$.
- RMS norm: $\|x\|_{RMS} = \sqrt{\frac{1}{d} \sum_i x_i^2}$.
- Fan-in = d_{in} ; fan-out = d_{out} .



1. First-Order Optimization

1.1 Gradient Descent (GD)

$$w_{t+1} = w_t - \eta g_t.$$

Assumptions for stability

- Quadratic loss $L = \frac{1}{2} \|Xw - y\|^2$:

$$\eta \leq \frac{1}{\lambda_{\max}(X^\top X)}.$$

- General smooth losses require **L-smoothness**:

$$\eta < \frac{2}{L}.$$

Geometry (linear models)

- Updates lie in **column space** of X .
- GD converges to **minimum-norm interpolator** *only if*:
 - $w_0 \in \ker(X)$ (e.g., $w_0 = 0$),
 - stable LR,
 - interpolation achieved.

Implicit bias

- Small singular directions of X updated slowly → GD mimics ridge.
- Exact equivalence holds in **gradient flow**, approximate for discrete GD.

Lecture 2



1.2 Stochastic Gradient Descent (SGD)

Unified Cheat Sheet

$$w_{t+1} = w_t - \eta g_{b_t}.$$

Key facts

- Noise enables escaping saddles.
- Converges with Robbins–Monro LR or constant LR in interpolation.
- In general NNs, updates lie in span of **gradient backpropagated features**, not raw data.

Convergence intuition (slides)

Decompose update in SVD basis:

$$\mathbb{E} \|z_{t+1}\|^2 \leq (1 - c\eta + c'\eta^2) \|z_t\|^2.$$

Requires small η and full-rank conditions.

Lecture 2

1.3 Momentum

$$v_{t+1} = \beta v_t + (1 - \beta) g_t, \quad w_{t+1} = w_t - \eta v_{t+1}.$$

Effects under curvature (Hessian eigenvalue λ)

Update along eigen-direction behaves like:

$$[1 - \eta\lambda + \beta].$$

- Momentum **accelerates** directions with **moderate/large λ** .
- Does NOT** fix conditioning: the condition number remains unchanged.



Lecture 2

1.4 Adam / AdamW

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t, \quad v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2.$$

$$w_{t+1} = w_t - \eta \frac{m_t}{(1 - \beta_1^t)}$$

$$w_{t+1} = w_t - \eta \frac{v_t}{\sqrt{v_t/(1 - \beta_2^t)} + \epsilon}.$$

Notes

- Adaptive coordinate-wise learning rates improve conditioning.
- AdamW decouples weight decay → corrects Adam's generalization issues.
- Still may generalize worse than SGD in some regimes.

Lecture 3

1.5 Sign-SGD (norm-constrained view)

Solve:

$$\min_{\|\Delta w\|_\infty \leq \eta} \langle g_t, \Delta w \rangle \Rightarrow \Delta w = -\eta \text{sign}(g_t).$$

Interpretation: norm choice defines optimization geometry.

Lecture 5

2. Ridge Regression & Implicit Regularization

Ridge solution

$$w_\lambda = (X^\top X + \lambda I)^{-1} X^\top y = \sum_i \frac{v_i}{\sigma_i^2 + \lambda} u_i (v_i^\top y).$$

GD relationship

- Gradient flow = ridge with time-dependent λ (exact).
- Finite-step GD ≈ ridge if LR small.
- Early stopping suppresses small- σ components.

Lecture 2

3. NTK, Linearization, and Lazy vs Feature Learning

3.1 Local linearization

$$f(\theta + \Delta\theta) \approx f(\theta) + J(\theta)\Delta\theta, \quad J = \nabla_\theta f.$$

Assumptions for accuracy

$$\|\Delta f\| = \|J\Delta\theta\| \ll 1, \quad \|\theta_t - \theta_0\|_2 \ll \frac{1}{\sqrt{\text{width}}}.$$

Both conditions depend on how LR scales with width.

3.2 Neural Tangent Kernel (NTK)

$$K(x, x') = J(x, \theta_0)^\top J(x', \theta_0).$$

Infinite-width limit (standard parametrization):

- Training behaves like kernel regression → no learned features.

Finite width:

- NTK accuracy depends on LR and scaling.

Lecture 3

Lecture 4

3.3 Why wide nets ≠ automatically NTK

Lazy regime occurs if parameterization and LR scaling ensure:

$$\|\Delta f\| \ll 1.$$

Without LR shrinking, Xavier/He networks can learn features even at large width.

4. Initialization Principles

4.1 Xavier (tanh/sigmoid)

$$w_{ij} \sim \mathcal{N}\left(0, \frac{1}{d_{\text{in}}}\right).$$

4.2 He (ReLU)

$$w_{ij} \sim \mathcal{N}\left(0, \frac{2}{d_{\text{in}}}\right).$$

Purpose: keep activations & gradients $O(1)$; does not determine whether training is lazy — LR scaling

Lecture 4

5. Gradient Scaling With Width (Critical for μP)

For layer $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$:

- Each gradient element:

$$G_{ij} = O(1/\sqrt{d_{\text{in}}}).$$

- Total Frobenius norm:

$$\|G\|_F = O\left(\sqrt{\frac{d_{\text{out}}}{d_{\text{in}}}}\right).$$

Thus widening changes functional update size unless LR is scaled appropriately.

6. RMS→RMS Norm (Link Between Initialization & Update Geometry)

Definition

$$\|x\|_{RMS} = \sqrt{\frac{1}{d} \sum x_i^2}$$

Matrix induced norm:

$$\|A\|_{RMS \rightarrow RMS} = \max_{\|x\|_{RMS}=1} \|Ax\|_{RMS} = \sqrt{\frac{d_{in}}{d_{out}}} \|A\|_2.$$

(Identify depends on RMS definition in slides)

Significance

- Same scaling pattern appears in Xavier/He.
- μP uses RMS→RMS to ensure Δf remains $O(1)$ across widths.

Lecture 6

7. μP — Maximal Update Parameterization

Core principle

Scale LR, initialization, and weights so **function updates**, not weight updates, remain $O(1)$ as width $\rightarrow \infty$.

Key conditions (slides)

1. Activations $O(1)$.
2. Weight updates satisfy

$$\|\Delta W_t\|_{RMS \rightarrow RMS} = O(1).$$

3. Feature updates $\neq$ vanish (avoid NTK collapse).

Scaling rules (simplified but correct)

$$\eta_t \propto \frac{1}{d_{in}}, \quad W_t^{init} \propto \frac{1}{\sqrt{d_{in}}}.$$

Consequence

- Hyperparameters transfer across widths.
- Network learns features instead of entering lazy/NTK regime.

8. Second-Order Geometry Methods

8.1 Shampoo

Uses approximate matrix square-root/inverse to precondition gradients:

$$W_{t+1} = W_t - \eta(G_t G_t^\top + \epsilon I)^{-1/2} G_t (G_t^\top G_t + \epsilon I)^{-1/2}.$$

- Approximates second-order curvature using Kronecker structure.
- Memory-heavy; requires stabilizers ϵ .

Lecture 5

Lecture 7

8.2 Muon — Momentum + Direction Normalization

Step 1: Momentum

$$B_t = \mu B_{t-1} + G_t.$$

In minibatch=1, $G_t = uv^\top \rightarrow$ rank-1 $\Rightarrow$ efficient SVD-like updates.

Step 2: Normalize gradient direction (not curvature)

Normalize singular values via Newton–Schulz polynomial:

$$B_t = U \Sigma I \downarrow \rightarrow UIV^\top.$$

Reshape into symmetric (defining preconditioner) as $\tilde{O}_t \in \mathbb{R}^{d \times d}$

Step 3: Update

$$W_{t+1} = W_t - \eta \tilde{O}_t, \quad \tilde{O}_t \approx UIV^\top.$$

Important distinctions

- **Not** a Hessian-based method — Muon reshapes *gradient* geometry, not curvature.
- Does **not** fix condition number, but removes dominant singular-value distortion.
- Synergizes with μP for width-invariant updates.

Lecture 7

9. Unifying Principles (Deep Insights)

1. Optimization = Choosing a Geometry

Update rule = solution to:

$$\min_{\|\Delta \theta\|_N \leq \eta} \langle g, \Delta \theta \rangle$$

Different norms $\Rightarrow$ GD, Sign-SGD, Shampoo, Muon.

2. Initialization Sets Signal Scale; Optimizer Must Respect It

Xavier/He preserve activations. μP & Muon preserve **functional update magnitudes**, ensuring scale-consistent training across width.


```

\section{Notation}
\begin{itemize}[leftmargin=]
\item Parameters  $\mathbf{w} \in \mathbb{R}^d$ , layer weights  $\mathbf{W} \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ 
\item Loss  $\mathcal{L}(\mathbf{w})$ , sample loss  $\ell_i(\mathbf{w})$ , gradient  $\mathbf{g}_t = \nabla \mathcal{L}(\mathbf{w}_t)$ 
\item SVD:  $\mathbf{A} = U \Sigma V^\top$ ; fan-in =  $d_{\text{in}}$ , fan-out =  $d_{\text{out}}$ 
\item RMS norm:  $\|\mathbf{x}\|_{\text{RMS}} = \sqrt{\frac{1}{d} \sum_i x_i^2}$ 
\end{itemize}

% =====
\section{First-Order Optimization}

\subsection{Gradient Descent (GD)}
\begin{itemize}[leftmargin=]
\item  $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t$ .
\item \textbf{Stability assumptions:}
For quadratic loss,  $\eta \leq \frac{1}{\lambda_{\max}(X^\top X)}$ .
For general smooth loss, require  $\eta \leq \frac{2}{L}$  (Lipschitz gradient constant  $L$ ).
\item \textbf{Geometry (linear models):}
Updates lie in  $\text{col}(X)$ ; convergence to minimum-norm interpolator requires  $\mathbf{w}_0 \in \text{ker}(X)$ , stable  $\eta$ , and zero-loss interpolation.
\item \textbf{Implicit bias:}
Small-singular-value directions updated slowly  $\Rightarrow$  ridge-like behavior.
\end{itemize}

\subsection{Stochastic Gradient Descent (SGD)}
\begin{itemize}[leftmargin=]
\item  $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_{i_t}$ .
\item Noise aids saddle escape; convergence via Robbins–Monro or constant LR in interpolation.
In NNs, updates lie in span of backpropagated feature gradients.
\end{itemize}

\subsection{Momentum}
\begin{itemize}[leftmargin=]
\item  $\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1-\beta) \mathbf{g}_t, \mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{v}_{t+1}$ .
\item Eigen-dynamics for Hessian eigenvalue  $\lambda$ :
 $|1 - \eta(\lambda + \beta)| < 1$ . Momentum accelerates but does not fix conditioning.
\end{itemize}

\subsection{Adam / AdamW}
\begin{itemize}[leftmargin=]
\item  $\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1) \mathbf{g}_t, \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2) \mathbf{g}_t^2$ ,
 $\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t + \epsilon}}$ .
\end{itemize}

\subsection{Sign-SGD}
\begin{itemize}[leftmargin=]
\item  $\|\Delta \mathbf{w}_t\| \leq \eta \|\mathbf{g}_t\|$ .
 $\Delta \mathbf{w} = -\eta \text{sign}(\mathbf{g}_t)$ .
\item Illustrates how choice of update norm defines the optimizer's geometry.
\end{itemize}

% =====
\section{Ridge Regression \& Implicit Regularization}
\begin{itemize}[leftmargin=]
\item  $\mathbf{w}_\lambda = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$ 
 $= \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda} u_i(v_i^\top \mathbf{y})$ .
\item Gradient flow = exact ridge; discrete GD approximates ridge for small  $\eta$ .
Early stopping damps small- $\sigma$  components.
\end{itemize}

% =====
\section{NTK, Linearization, and Learning Regimes}

\subsection{Linearization}
\begin{itemize}[leftmargin=]
\item  $f(\theta + \Delta \theta) \approx f(\theta) + J(\theta) \Delta \theta, J = \nabla_\theta f$ .
\item Valid when  $\|\Delta \theta\| \ll 1$ ,
 $\|\Delta \theta\| \leq \frac{1}{\sqrt{L}}$ .
\end{itemize}

\subsection{Neural Tangent Kernel}
\begin{itemize}[leftmargin=]
\item  $K(\mathbf{x}, \mathbf{x}') = J(\mathbf{x}, \theta_0)^\top J(\mathbf{x}', \theta_0)$ .
\item Under standard parametrization and width  $\rightarrow \infty$ , parameters move little  $\Rightarrow$ 
\end{itemize}

\subsection{Wide Nets Do Not Automatically Enter NTK}
NTK regime arises only if learning-rate/parameterization scales so that  $\|\Delta \theta\| \ll 1$ .
Xavier/He ensure stable activations, not lazy training by themselves.

% =====
\section{Initialization (Variance Preservation)}
\subsection{Xavier}
\begin{itemize}[leftmargin=]
\item  $\mathbf{w}_{ij} \sim \mathcal{N}(0, \frac{1}{d_{\text{in}} + d_{\text{out}}})$ .
\end{itemize}
\subsection{He (ReLU)}
\begin{itemize}[leftmargin=]
\item  $\mathbf{w}_{ij} \sim \mathcal{N}(0, \frac{2}{d_{\text{in}}})$ .
\end{itemize}
Goal: keep activations/gradients  $\mathcal{O}(1)$ ; LR scaling (not initialization alone)

```

